

Reporte Académico Final

Sistema de Pedidos, Documentación y justificación del diseño lógico

Nombre: Eric Eduardo Romero Alfaro

Matrícula: AL03029612

Materia: Fundamentos de Programación

Organización analizada: Negocio de comida mexicana

Área de impacto: Atención al cliente y toma de pedidos

Fecha de entrega: 25/09/2026

1. Contexto y problemática de negocio

Este proyecto desarrolla un sistema en Python para apoyar la operación diaria de un negocio de comida a domicilio. Está pensado para que lo use el encargado o mesero del restaurante (el "operador"), no el cliente final: es una herramienta interna para registrar pedidos, consultar información y llevar un cierre del día.

Antes de este sistema, los pedidos se registraban de forma manual (muy incomodo), lo que aumentaba el riesgo de perder información, cometer errores en los cálculos de subtotal, envío y total, y no tener un registro claro de qué pedidos se entregaron, cuáles quedaron pendientes de pago o cuáles no se pudieron atender por estar fuera de horario. El área de impacto es la atención al cliente y el control interno de pedidos.

2. Reglas de negocio

- Solo se reciben pedidos entre las 8:00 y las 16:00 horas (formato de 24 horas). Fuera de ese horario, el pedido se guarda en una lista de "pedidos diferidos" para atenderse al día siguiente.
- El costo de envío es de \$0 si el cliente está dentro de la colonia de cobertura, y de \$40 si está fuera.
- El pago puede ser en efectivo o por transferencia. Si es por transferencia y no se confirma en el momento, el pedido se guarda en una lista de "pendientes de pago".
- Cada pedido registrado genera un recibo individual (archivo de texto) con la fecha, el cliente, la dirección, el detalle de los platos pedidos y el total.
- Al cerrar el día, el sistema puede generar un reporte con el total vendido y las listas de pedidos diferidos y pendientes de pago.

2.1 Reglas de validación de datos

Además de las reglas propias del negocio, el sistema valida que los datos capturados tengan sentido, para evitar registros inconsistentes:

- La fecha debe ser válida: el día entre 1 y 31, el mes entre 1 y 12, y el año un número positivo.
- La cantidad de platos por pedido debe ser de al menos 1 y máximo 10.
- El precio de cada plato no puede ser un número negativo.
- Las respuestas de sí/no (colonia, forma de pago, confirmaciones) solo aceptan "s" o "n" el sistema vuelve a preguntar hasta recibir una respuesta válida.

3. Arquitectura de la solución

3.1 Modelo Entrada – Proceso – Salida

Entrada	Proceso	Salida
Nombre del operador	Guardar el nombre y mostrarlo en los encabezados del sistema	Mensaje de bienvenida en pantalla
Día, mes y año	Validar que sean valores posibles (día 1-31, mes 1-12, año positivo) y guardarlos como tupla	Fecha lista para integrarse en los archivos
Opcion del menu (1-7)	Decidir qué función ejecutar segun la opcion elegida	Menu mostrado nuevamente o cierre del programa
Datos del pedido (cliente, hora, direccion, cantidad y detalle de platos, colonia, forma de pago)	Validar horario y cantidades, calcular subtotal, envío y total, confirmar el pago	Total a pagar y recibo del pedido guardado en un archivo .txt
Nombre del archivo a leer o escribir	Abrir el archivo, leer o anexar contenido, controlando errores	Contenido mostrado en pantalla o archivo actualizado
Total del día, pedidos diferidos y pendientes	Reunir la información acumulada durante la sesion	Reporte de cierre .txt con el resumen del día

4. Justificación de los 10 requerimientos

La siguiente tabla documenta cómo el programa cumple cada uno de los diez requerimientos del proyecto, señalando la función donde se puede verificar la evidencia.

#	Requerimiento	Cómo se cumple	Evidencia en el código
1	Identificación de usuario	Se solicita el nombre del operador al iniciar el programa, antes de mostrar cualquier menú.	función bienvenida()
2	Bienvenida dinámica con operadores de cadena	El nombre capturado se concatena con "+" dentro de un encabezado centrado con .center().	función encabezado()
3	Pantalla de carga (max 5 s)	Un ciclo For de 5 vueltas imprime puntos suspensivos crecientes y pausa 1 segundo por vuelta.	función pantalla_carga()
4	Menú principal como matriz, controlado por While	Las opciones se guardan en una lista de listas, un ciclo While las muestra y procesa hasta elegir salir.	matriz_menu, imprimir_menu(), ciclo While principal
5	Control de inactividad (10 min, con For)	Se compara el tiempo entre interacciones con time.time(), si pasan 10 min, un ciclo For da 2 intentos para confirmar si se continúa.	función revisar_inactividad()
6	Fecha estructurada como tupla	Día, mes y año se validan y se guardan como tupla (fecha = día, mes, año), se reutiliza en cada archivo generado.	función pedir_fecha()
7	Persistencia en archivos de texto (lectura y escritura)	Se leen 4 archivos de consulta (menú, colonias, clientes, historial) y se generan/actualizan archivos de recibo y reporte de cierre.	leer_archivo(), consultar_menu(), escribir_archivo(), registrar_pedido(), reporte_cierre()
8	Control de excepciones (try-except)	Se capturan ValueError (datos no numéricos), KeyError (opción de archivo	pedir_fecha(), registrar_pedido(),

		inválida), FileNotFoundError, PermissionError y OSError (nombre de archivo inválido).	leer_archivo(), escribir_archivo(), reporte_cierre()
9	Depuración con PDB, documentada	Se utilizó el módulo pdb para revisar el comportamiento del programa y corregir fallas de lógica encontradas durante las pruebas.	Ver sección "Evidencia de depuración con PDB"
10	Comentarios de calidad	Cada función tiene un comentario que explica su propósito, las reglas de negocio están documentadas en un docstring dedicado.	Comentarios a lo largo de main.py, docstring sobre registrar_pedido()

5. Evidencia de depuración con PDB

Durante el desarrollo se usó el módulo pdb (Python Debugger) para revisar el comportamiento del programa paso a paso y encontrar fallas de lógica que no eran evidentes solo con leer el código. Se colocó `pdb.set_trace()` dentro del ciclo principal del menú, justo después de llamar a `revisar_inactividad()`, y se usaron los comandos `p` (para imprimir el valor de una variable) y `c` (para continuar la ejecución).

Falla encontrada: TypeError al reiniciar por inactividad

Contexto: cuando pasan 10 minutos de inactividad y el operador responde que no quiere continuar, `revisar_inactividad()` debe regresar la señal de texto "inicio" para que el programa vuelva a la pantalla de bienvenida. En el código que se estaba probando, el ciclo principal asignaba ese resultado directamente a `ultima_interaccion`, sin distinguir si era un número (un nuevo tiempo) o el texto "inicio".

```
{Cual es tu nombre?Eric
-----
Bienvenido al sistema para pedir Eric
-----
cargando.
cargando..
cargando...
cargando....
cargando.....
Día del pedido: 24
Mes del pedido: 09
Año del pedido: 2026
1.Registrar pedido
2.Ver el menu de platos
3.Leer archivo
4.Escribir o anexar archivo
5.Revisar reporte de cierre
6.Ver pendientes/diferidos
7.Salir
¿Que quieres hacer?: 1
Pasaron 10 minutos de inactividad, quieres seguir usandolo? (s/n): n
cargando.
cargando..
cargando...
cargando....
cargando.....
> c:\users\erice\workspace\fundamentos_de_programacion\semana7\main.py(265)<module>()
-> pdb.set_trace()
(Pdb) p ultima_interaccion
'inicio'
(Pdb) c
Nombre del cliente: Amaris
Hora del pedido: 12
Direccion: P arbol
Cuantos platos?1
```

EL TEXTO EN TERMINAL PUEDE SER DIFERENTE AL ACTUAL PORQUE SE DETECTÓ LA FALLA ANTES DE HACER LOS ÚLTIMOS CAMBIOS Y DEJARLO MÁS PRESENTABLE

el programa reinicia por inactividad. Al inspeccionar la variable con (Pdb) p ultima_interaccion, su valor es el texto 'inicio' en vez de un número.

```
-> pdb.set_trace()
(Pdb) p ultima_interaccion
'inicio'
(Pdb) c
Nombre del cliente: Amaris
Hora del pedido: 12
Direccion: P arbol
Cuantos platos?1
Nombre del plato: enchiladas
Precio del plato:75
Esta en la colonia? (s/n)s
total a pagar: 75.0
No se guardo el archivo, no existe la ruta
Pago en efectivo (s/n)s
pedido enviado a cocina
1.Registrar pedido
2.Ver el menu de platos
3.Leer archivo
4.Escribir o anexar archivo
5.Revisar reporte de cierre
6.Ver pendientes/diferidos
7.Salir
¿Que quieres hacer?: 2
Traceback (most recent call last):
  File "c:\Users\erice\workspace\fundamentos_de_programacion\semana7\main.py", line 264, in <module>
    ultima_interaccion = revisar_inactividad(ultima_interaccion)
  File "c:\Users\erice\workspace\fundamentos_de_programacion\semana7\main.py", line 135, in revisar_inactividad
    tiempo_pasado = ahora - ultima_interaccion
                        ~~~~~^~~~~~
TypeError: unsupported operand type(s) for -: 'float' and 'str'
PS C:\Users\erice\workspace>
```

En la siguiente vuelta del menú, revisar_inactividad() intenta restar ese texto a un número (ahora ultima_interaccion) y el programa se detiene con TypeError: unsupported operand type(s) for -: 'float' and 'str'.

```

while opcion != "7":
    imprimir_menu(matriz_menu)
    opcion = input("¿Que quieres hacer?: ")

    ultima_interaccion = revisar_inactividad(ultima_interaccion)
    pdb.set_trace()

    if opcion == "1":
        total_pedido = registrar_pedido(Fecha)
        total_día = total_día + total_pedido
    elif opcion == "2":
        consultar_menu()

```

línea del ciclo principal donde ocurría el problema — `ultima_interaccion` se sobrescribía directamente con lo que regresara `revisar_inactividad()`, sin distinguir si era un número o la señal de texto "inicio".

Corrección aplicada: se separó el valor de retorno de `revisar_inactividad()` en una variable intermedia (`resultado`). Solo cuando `resultado` no es la cadena "inicio" se actualiza `ultima_interaccion` con un nuevo tiempo (`time.time()`), cuando sí lo es, se ejecuta el reinicio (nueva bienvenida y nueva fecha) y continúe salta el resto del ciclo, evitando que un texto quede guardado donde el programa espera un número.

7. Conclusiones

7.1 Aprendizajes

- Diferencia práctica entre tuplas (para datos que no deben cambiar, como la fecha) y listas (para datos que sí cambian, como las colas de pedidos).
- Cuándo repetir una pregunta con un ciclo (`while True` con `break`) en vez de solo cancelar la operación ante un dato inválido.
- El valor de extraer código repetido a una función propia (como `preguntar_sn()`) para no duplicar la misma validación varias veces.
- El propósito de `try-except`: evitar que un error de un solo dato o archivo detenga todo el programa.

7.2 Dificultades superadas

- Errores de rutas relativas al ejecutar el programa desde una carpeta distinta a la del proyecto.
- Errores de indentación comunes al organizar el código en funciones.

- Diseñar el control de inactividad entendiendo que un programa de consola no puede "vigilar" el tiempo mientras espera una respuesta del usuario (input), por lo que se optó por medir el tiempo transcurrido entre una interacción y la siguiente.

7.3 Mejoras futuras

- Detectar inactividad en tiempo real usando hilos (threading), en vez de medirla solo entre una interacción y la siguiente.
- Distinguir entre el total "registrado" y el total "confirmado" (ya cobrado) en el reporte de cierre.
- Aceptar la hora en formato de 12 horas (AM/PM) además del formato de 24 horas.
- Reemplazar los archivos de texto por una base de datos si el sistema creciera en volumen de pedidos.